

Predicting CVSS and CWE from CVE Descriptions with Fine-Tuned Small Language Models

Tara Dixit

Jessie Dong

Summer Li

Stanford University

{taradixit, jessiedong, summerli}@stanford.edu

Abstract

Since April 2026, the U.S. National Vulnerability Database enriches only high-priority CVEs, so most new vulnerabilities arrive with a text description and no NIST-assigned CVSS vector, severity or weakness class. We ask how well a model can supply these fields from the description alone. Using a pinned NVD snapshot, we train on CVEs from 2022–2023 and test on all 36,325 usable CVEs from 2024. We compare TF-IDF classifiers, a 3.8B-parameter language model (Phi-3.5-mini) used zero-shot, and the same model fine-tuned with LoRA, and we compare predicting the CVSS score directly with computing it from the predicted vector. The fine-tuned model is best on every field: 69.9% severity accuracy, 70.9% CWE accuracy, a score MAE of 0.85, and an exact CVSS vector on 48.8% of test CVEs, ahead of TF-IDF logistic regression by +2.8 to +4.7 points, with paired bootstrap intervals that exclude zero. Computing the score from the vector helps logistic regression but trades precision on the CRITICAL tier for recall, and the fine-tuned model learns to keep its own score and vector consistent on 99.8% of CVEs. The zero-shot model trails both. We release code, configuration and per-CVE predictions.

1 Introduction

When a software vulnerability is disclosed, it receives a CVE identifier and a short text description. The description alone does not tell a security team how urgently to act. For that, teams rely on structured *enrichment*: a CVSS v3.1 vector that records how the flaw is exploited and what it affects, the base score (0–10) and severity tier that follow from that vector, and a CWE category for the type of weakness (FIRST, 2019). Scanners, patch managers, and incident responders sort their queues by these fields.

For most CVEs, the National Vulnerability Database (NVD) at NIST supplied this enrichment. It has not kept pace. A backlog formed in early 2024, and on 15 April 2026 NIST announced that it will now enrich only CVEs in CISA’s Known Exploited Vulnerabilities catalog, CVEs in software the federal government uses, and critical software under Executive Order 14028. Everything else is marked “Lowest Priority – not scheduled for immediate enrichment” (National Institute of Standards and Technology, 2026). For most new CVEs, defenders now get the description and whatever score the reporting organization chose to supply.

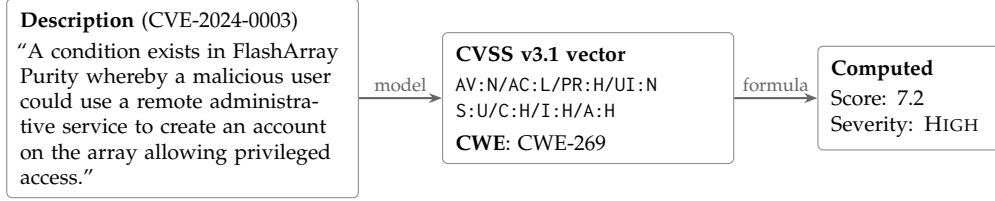


Figure 1: **From a CVE description to a full NVD record.** A model reads the description and predicts the eight CVSS components and the CWE. In the formula variant, the score and severity are computed from the vector with the CVSS v3.1 equations, so the three always agree.

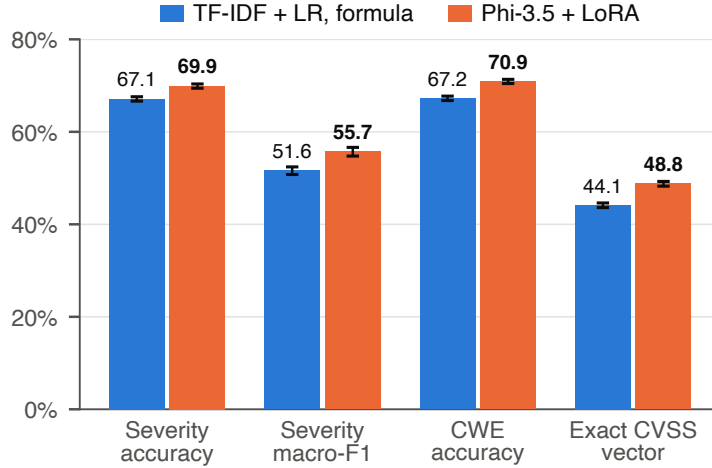


Figure 2: **Fine-tuned Phi-3.5 beats TF-IDF on every field.** Full 2024 test set (36,325 CVEs). TF-IDF uses logistic regression with formula scoring. Error bars are 95% bootstrap intervals.

This paper asks how well a model can fill that gap from the description alone, using only public data. We call this task **CVE enrichment**: given only the description, predict the eight CVSS v3.1 vector components, the base score, the severity tier, and the CWE (Figure 1). We compare TF-IDF classifiers, a small instruction-tuned language model used zero-shot, and the same model fine-tuned with LoRA.

Two design choices shape the study. First, we split by time. We train on CVEs from 2022–2023 and test on every usable CVE from 2024, so a model never sees the period it is scored on. Random splits, which several public models use, can put near-duplicate descriptions from the same product and month on both sides of the split. Second, we treat the score as something to *compute*, not something to predict. The CVSS base score and severity are fixed functions of the eight vector components. Following Elbaz et al. (2020) and Shahid and Debar (2021), we predict the components and apply the CVSS formula. We then measure what this buys over predicting the score directly, and whether a language model’s own score agrees with the vector it writes.

On all 36,325 test CVEs, the fine-tuned model is the best system on every field (Figure 2). It reaches 69.9% severity accuracy and 70.9% CWE accuracy, and it predicts the exact CVSS vector for 48.8% of CVEs, ahead of TF-IDF logistic regression by +2.8 to +4.7 points. Its scores agree with its own vectors on 99.8% of CVEs. The same model used zero-shot agrees with itself on only 3.4%. Fine-tuning took 2.5 hours.

Our contributions:

- **Benchmark.** A leakage-free temporal split for CVE enrichment: 50,545 training and validation CVEs from 2022–2023 and 36,325 test CVEs from 2024, from a pinned NVD snapshot, with a CVSS calculator that reproduces all 86,870 NVD base scores exactly.
- **Comparison.** Classical, zero-shot, and fine-tuned models on every enrichment field, with 95% bootstrap confidence intervals.
- **Analysis.** Direct versus formula-based scoring, and the self-consistency of language-model outputs.
- **Release.** Code, configuration, and per-CVE predictions for every system.

2 Related Work

CVSS vector prediction. Elbaz et al. (2020) predicted each CVSS metric from a bag-of-words description with linear models and derived the score from the predicted vector, with an emphasis on explainable decisions at disclosure time. CVSS-BERT (Shahid and Debar, 2021) trained one BERT classifier per CVSS v3.1 metric and computed the score with the specification’s equations. Marchiori et al. (2025) compared general-purpose LLMs with embedding-based classifiers for computing CVSS vectors and found the embedding methods stronger on the more subjective impact metrics (confidentiality, integrity, availability). We follow the predict-the-vector design of these works and add a fine-tuned small language model, a strict temporal split, and a direct comparison with score regression.

Severity and CWE classification. VLAI (Bonhomme and Dulaunoy, 2025) fine-tunes RoBERTa on more than 600,000 vulnerabilities to predict the severity tier. Its training set spans CVEs from many years, including 2024, so it is not comparable on our test set. We discuss this in Section 3. For CWE assignment, Aota et al. (2020) used bag-of-words features with random forests, and Mosievskiy (2026) reports that a 125M-parameter RoBERTa is competitive with much larger LLMs.

LLMs for vulnerability triage. Talibzade et al. (2025) found that TF-IDF classifiers beat general-purpose LLMs with generic prompts on CWE and severity assignment, and attribute this to the schematic wording of CVE descriptions. CVE-LLM (Ghosh et al., 2025) fine-tunes an LLM with a domain ontology to evaluate medical-device vulnerabilities. Torkamani et al. (2025) use LLMs to assign CWEs and severity in a triage tool. For a related task, predicting exploitation, Roytman (2025) reports that frontier LLMs fall well short of the statistical EPSS model and cost far more to run at scale. Industry systems such as VulnWatch (Databricks, 2025) prioritize vulnerabilities with LLMs on hosted infrastructure. Our setting differs in two ways: everything runs locally, and every output is evaluated on vulnerabilities disclosed after the training data.

3 Task and Data

Task. The input is the English description of a CVE. The outputs are the eight CVSS v3.1 base metrics (attack vector, attack complexity, privileges required, user interaction, scope, and the confidentiality, integrity, and availability impacts), the base score, the severity tier (CRITICAL, HIGH, MEDIUM, LOW), and the CWE identifier. Figure 1 shows one example.

Source. We take NVD records from the Fraunhofer FKIE mirror of the NVD JSON feeds (Fraunhofer FKIE, 2026), pinned to release v2026.10.05-000016. NVD records are revised over time, so we pin the release to make every number in this paper reproducible.

Table 1: **Data splits.** Severity columns give the share of CVEs (%) in each tier.

Split	CVEs	Critical	High	Medium	Low	CWEs
Train (2022–23)	40,436	15.4	39.3	43.7	1.7	464
Validation (2022–23)	10,109	15.4	39.3	43.7	1.7	293
Test (2024)	36,325	13.2	36.2	48.6	1.9	512

Filtering. We keep a CVE if it has an English description of at least 20 characters, a CVSS v3.x base metric with a severity tier, and a specific CWE (we drop the placeholders NVD-CWE-noinfo and NVD-CWE-Other). When a CVE has several CVSS entries, we prefer NIST’s primary score. This leaves 86,870 CVEs from 2022–2024. Of these, 776 use CVSS v3.0. The rest use v3.1, and the base-score equations of the two versions agree on all of them.

Temporal split. CVEs with 2022 and 2023 identifiers form the training and validation pool (50,545 CVEs), split 80/20 at random with severity stratification into 40,436 training and 10,109 validation CVEs. All 36,325 usable CVEs with 2024 identifiers form the test set. Validation data is used only to choose hyperparameters and checkpoints. Table 1 gives the label distribution.

Label distribution. LOW is under 2% of every split. CWEs follow a long tail: the training set contains 464 distinct CWEs and the test set 512. The 100 most frequent training CWEs cover 91.7% of test CVEs.

CVSS calculator. We implemented the CVSS v3.1 base-score equations (FIRST, 2019), including the specification’s floating-point-safe round-up. Applied to the true vectors, it reproduces the NVD base score and severity for all 86,870 CVEs. Any error in a formula-based score therefore comes from the predicted vector, not from the calculator.

Excluded baseline. VLAI (Bonhomme and Dulaunoy, 2025) is a public severity classifier. Its training data, the CIRCL/vulnerability-scores dataset, uses a random split over all years. Rows we sampled from its training split include CVEs from 2024, 2025, and 2026. Scoring it on our 2024 test set would measure memorization as much as generalization, so we leave it out.

4 Methods

4.1 Direct and Formula Scoring

Every system below predicts the eight components and the CWE. We then produce the score and severity in one of two ways.

Direct. A separate model predicts the score (regression) and the severity (classification), or a language model writes them itself.

Formula. We apply the CVSS v3.1 equations to the predicted components. Score, severity, and vector then agree by construction.

For the TF-IDF systems, the direct score comes from a single ridge regression shared by all three classifiers. Their direct score errors are therefore identical, and we report them once.

4.2 TF-IDF Baselines

We represent each description with TF-IDF over unigrams and bigrams (sublinear term frequency, minimum document frequency 2, at most 50,000 features). We keep stop words, since words such as “not” and “without” change the meaning of a description. We train one classifier per output with scikit-learn (Pedregosa et al., 2011):

- **Logistic regression (LR).** We choose $C \in \{0.3, 1, 3, 10\}$ on validation accuracy, separately for severity, CWE, and the components (one C shared by all eight).
- **Class-balanced LR.** The same, with class weights inversely proportional to class frequency.
- **Random forest (RF).** 300 trees for severity and CWE and 100 trees per component, otherwise default settings. We did not tune it.
- **Ridge regression** for the direct score, with $\alpha \in \{0.3, 1, 3, 10\}$ chosen on validation MAE.

The CWE classifiers learn the 100 most frequent training CWEs plus an OTHER class. We score them against the true CWE, so an OTHER prediction always counts as wrong.

4.3 Phi-3.5-mini

We use Phi-3.5-mini-instruct, a 3.8B-parameter decoder (Abdin et al., 2024). Its training-data cutoff is October 2023, before our test period.

Fine-tuning. We fine-tune with LoRA (Hu et al., 2022) in MLX (Hannun et al., 2023) on a single Apple M5 Pro with 64 GB of unified memory. Adapters of rank 16 (scale 20, dropout 0.05) sit on the last 16 transformer layers: 12.6M trainable parameters, 0.329% of the model. We train for 5,000 steps at batch size 8, which is one pass over the training set, with a learning rate of 10^{-4} , 100 warm-up steps, and cosine decay to 10^{-5} . The loss covers only the answer tokens. We drop the 561 training examples (1.4%) longer than 768 tokens to bound memory. Test CVEs are never dropped. The target is a single JSON object that lists the eight components first, then the CWE, the score, and the severity. The model therefore commits to a vector before it writes a score. We evaluate the checkpoints at steps 1,000, 2,000, ..., 5,000 on the same 1,000 validation CVEs and keep the one with the best severity accuracy (formula variant). Decoding is greedy.

Zero-shot. The same base model, without an adapter, receives the description and a prompt listing every field and its allowed values (Appendix A). Generating for every test CVE is slow for verbose zero-shot output, so we evaluate it on a random sample of 2,000 test CVEs and rescore every other system on the same sample.

4.4 Metrics

For severity we report accuracy and macro-F1 over the four tiers. Macro-F1 weights LOW as much as MEDIUM and exposes failures on the rare tiers that accuracy hides. For CWE we report top-1 accuracy, for the score the mean absolute error (MAE), and for the vector the per-component accuracy, the mean fraction of components correct (*partial match*), and the fraction of CVEs with all eight correct (*exact match*). A language-model output that is not valid JSON counts as wrong on every field and is left out of the score MAE. We report the parse rate separately. We give 95% confidence intervals from 1,000 bootstrap resamples of test CVEs.

Table 2: **Fine-tuned Phi-3.5 is best on every metric.** All systems on the full 2024 test set (36,325 CVEs). Values are percentages except score MAE. $\pm$ gives the half-width of the 95% bootstrap interval. Bold marks the best value in each column. The TF-IDF direct systems share one ridge regression, so their score MAE is identical.

System	Severity		CWE	Score	Vector	
	Acc.	Macro-F1	Acc.	MAE $\downarrow$	Partial	Exact
TF-IDF + LR, direct	67.1 $\pm$ 0.5	47.9 $\pm$ 0.6	67.2 $\pm$ 0.5	0.98 $\pm$ 0.01	86.9 $\pm$ 0.2	44.1 $\pm$ 0.5
TF-IDF + LR, formula	67.1 $\pm$ 0.5	51.6 $\pm$ 0.8	67.2 $\pm$ 0.5	0.94 $\pm$ 0.01	86.9 $\pm$ 0.2	44.1 $\pm$ 0.5
TF-IDF + LR balanced, direct	65.7 $\pm$ 0.5	51.9 $\pm$ 0.8	67.2 $\pm$ 0.5	0.98 $\pm$ 0.01	86.0 $\pm$ 0.2	40.5 $\pm$ 0.5
TF-IDF + LR balanced, formula	67.1 $\pm$ 0.5	53.9 $\pm$ 0.9	67.2 $\pm$ 0.5	0.98 $\pm$ 0.01	86.0 $\pm$ 0.2	40.5 $\pm$ 0.5
TF-IDF + RF, direct	66.9 $\pm$ 0.5	47.1 $\pm$ 0.6	65.2 $\pm$ 0.5	0.98 $\pm$ 0.01	86.1 $\pm$ 0.2	41.8 $\pm$ 0.5
TF-IDF + RF, formula	64.0 $\pm$ 0.5	47.8 $\pm$ 0.6	65.2 $\pm$ 0.5	1.03 $\pm$ 0.01	86.1 $\pm$ 0.2	41.8 $\pm$ 0.5
Phi-3.5 + LoRA, direct	69.9 $\pm$ 0.5	55.7 $\pm$ 0.9	70.9 $\pm$ 0.5	0.85 $\pm$ 0.01	87.9 $\pm$ 0.2	48.8 $\pm$ 0.5
Phi-3.5 + LoRA, formula	69.9 $\pm$ 0.5	55.7 $\pm$ 0.9	70.9 $\pm$ 0.5	0.85 $\pm$ 0.01	87.9 $\pm$ 0.2	48.8 $\pm$ 0.5

5 Results

5.1 Main Results

Table 2 gives all systems on the 36,325 test CVEs. The fine-tuned Phi-3.5 model leads on every metric. It reaches 69.9% severity accuracy, 70.9% CWE accuracy, a score MAE of 0.85, and an exact vector on 48.8% of CVEs.

Because every system is scored on the same CVEs, we can compare them directly with a paired bootstrap. Relative to TF-IDF logistic regression with formula scoring, the fine-tuned model changes severity accuracy by +2.8 points (95% CI +2.4 to +3.1), macro-F1 by +4.1 (+3.0 to +5.2), CWE accuracy by +3.7 (+3.4 to +4.0), score MAE by -0.09 (-0.10 to -0.08), and exact-vector rate by +4.7 (+4.3 to +5.0). No interval includes zero. The gains are consistent but modest, and TF-IDF remains a strong baseline.

5.2 Direct vs. Formula Scoring

For logistic regression, computing the score from the predicted vector helps. Score MAE falls from 0.98 to 0.94 and macro-F1 rises from 47.9% to 51.6%, while accuracy stays at 67.1%. The gain comes from the two rare tiers (Table 3): LOW F1 rises from 2.2% to 10.9%, and CRITICAL F1 from 51.4% to 56.1%, as CRITICAL recall rises from 50.7% to 73.3%.

This is a trade, not a free gain. The formula system predicts CRITICAL for 7,755 CVEs where NVD has 4,800, and its precision on that tier falls from 52.1% to 45.4%. The formula system scores 17.7% of test CVEs at exactly 9.8, the score of the most common critical vector (network, low complexity, no privileges, no interaction, high impact), against 10.5% in NVD, and its mean signed score error is +0.33, against +0.08 for ridge regression. For a triage queue, catching more critical CVEs at the cost of more false alarms may be the right trade, but it is a choice, not an improvement on every axis.

For the random forest, the formula hurts: accuracy falls from 66.9% to 64.0% and MAE rises from 0.98 to 1.03. Its component predictions are less accurate, and the formula passes those errors through.

The fine-tuned model shows almost no difference between the two modes. It wrote a score that matches the CVSS formula for its own vector on 99.8% of test CVEs, and a severity that matches its own score on 100.0% (Table 5). Trained on targets that list the components first, it behaves as if it applies the formula to its own vector.

Table 3: **LOW is the hardest tier for every system.** Recall and F1 (%) by true severity tier on the 2024 test set.

System	Critical		High		Medium		Low	
	Rec.	F1	Rec.	F1	Rec.	F1	Rec.	F1
TF-IDF + LR, direct	50.7	51.4	62.2	61.1	77.9	77.1	1.1	2.2
TF-IDF + LR, formula	73.3	56.1	62.6	62.6	71.3	76.8	6.2	10.9
TF-IDF + LR bal., formula	57.1	52.8	61.9	62.2	75.6	77.2	19.2	23.2
Phi-3.5 + LoRA, formula	76.9	59.7	63.6	64.6	75.1	79.0	11.4	19.6

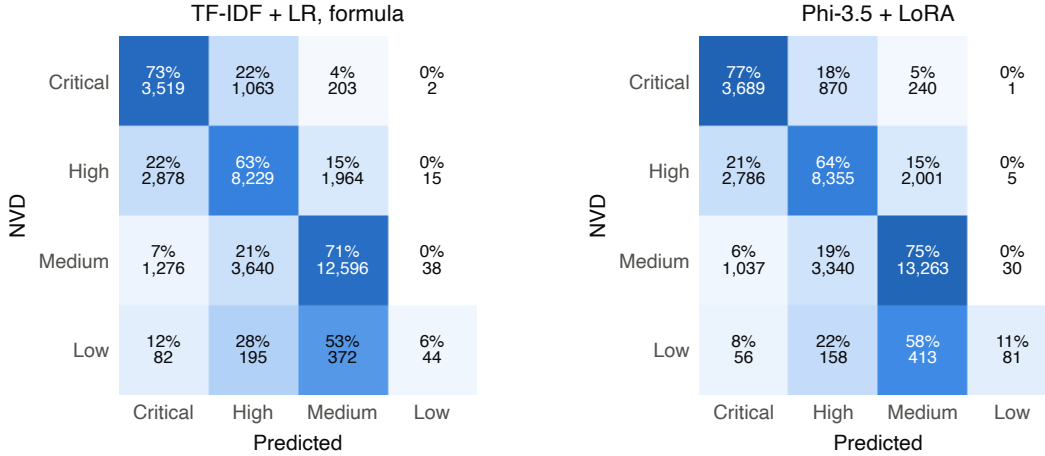


Figure 3: **Most severity errors fall between adjacent tiers.** Confusion matrices on the 2024 test set, normalized by true tier. Left: TF-IDF logistic regression with formula scoring. Right: fine-tuned Phi-3.5.

5.3 Error Analysis

Severity tiers. LOW remains the hardest tier for every system (Table 3). It is under 2% of the training data, and both systems most often label a true LOW CVE as MEDIUM. Class-balanced weights raise LOW recall to 19.2% for TF-IDF. The fine-tuned model, trained without rebalancing, reaches 11.4%. Figure 3 shows that most remaining confusion is between adjacent tiers.

CVSS components. Figure 4 gives accuracy per component. Attack complexity, scope, attack vector and user interaction are above 90% for both systems. Descriptions usually state them outright. Privileges required is the hardest (76.5% for TF-IDF, 78.8% fine-tuned), followed by the impact components. The fine-tuned model’s largest gains are on confidentiality (82.9% to 84.8%), integrity (82.9% to 84.5%) and privileges required, the components that depend least on surface keywords. [Marchiori et al. \(2025\)](#) found that general-purpose LLMs fall behind embedding-based classifiers on the impact components. After fine-tuning, those are among the components where our model gains most over TF-IDF.

Rare CWEs. The TF-IDF classifiers can only output the 100 most frequent training CWEs, so they score 0% on the 3,023 test CVEs whose CWE is rarer. The fine-tuned model writes the CWE as text and is right on 7.0% of them (Figure 5). Common, explicitly named weaknesses such as cross-site scripting and SQL injection are close to solved by both systems.

Score distribution. Ridge regression produces a smooth spread of scores that NVD never assigns, and it rarely predicts the extremes (Figure 6). The fine-tuned model’s scores sit

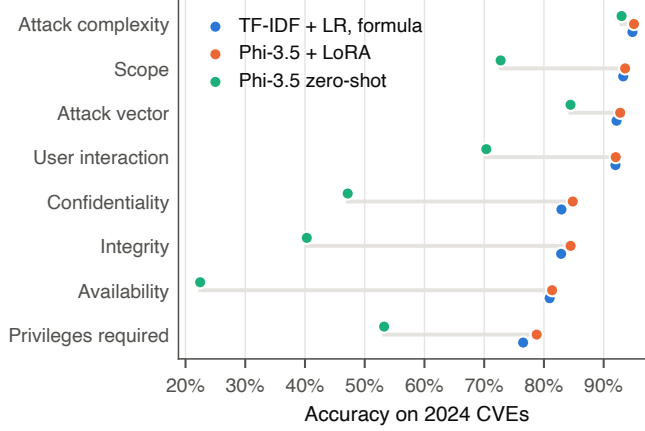


Figure 4: **Fine-tuning gains most on privileges, confidentiality, and integrity.** Accuracy per CVSS component on 2024 CVEs. TF-IDF and the fine-tuned model are scored on the full test set, the zero-shot model on its 2,000-CVE sample.

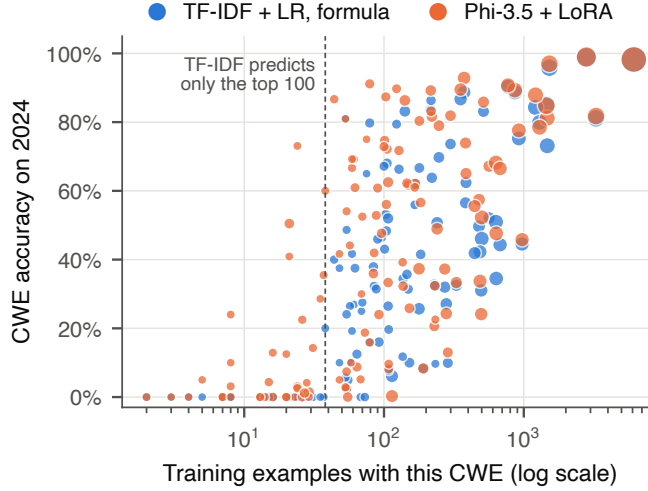


Figure 5: **Only the fine-tuned model can name rare CWEs.** CWE accuracy on the 2024 test set against the number of training examples with that CWE. Each point is a CWE with at least 20 test CVEs. Point size grows with test count.

on the same discrete values as NVD’s, but it predicts the most common critical score, 9.8, for 20.0% of test CVEs, where NVD assigns it to 10.5%. It predicts CRITICAL for 7,568 test CVEs, against 4,800 in NVD, with 48.7% precision, and like formula-scored logistic regression it scores high on average (mean signed error +0.33).

5.4 Zero-Shot Baseline

Without fine-tuning, the same base model is far weaker (Table 4). On 2,000 random test CVEs it returned valid JSON for 99.7%, but the severity it writes is biased upward: it labels 62.4% of them CRITICAL, against 13.3% in NVD, and it predicts MEDIUM only 9 times although 47.5% of these CVEs are MEDIUM. Its written severity is right on 20.8% of CVEs. Its outputs also contradict themselves: the score it writes matches the formula for its own vector on only 3.4% of CVEs, and the severity it writes matches its own score on 37.0% (Table 5). Computing severity from its vector removes the inconsistency and raises severity

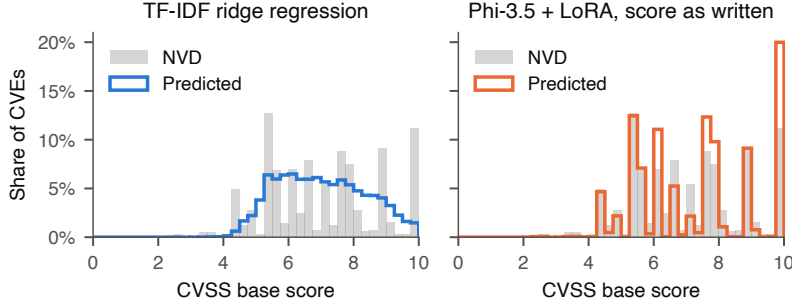


Figure 6: **Ridge regression smooths scores, and the fine-tuned model over-predicts 9.8.** Predicted against NVD base scores on the 2024 test set, in bins of 0.25.

Table 4: **Zero-shot Phi-3.5 trails both trained systems.** Comparison on the same random sample of 2,000 test CVEs. Every system is scored on exactly these CVEs.

System	Severity		CWE	Score	Vector	
	Acc.	Macro-F1	Acc.	MAE ↓	Partial	Exact
TF-IDF + LR, formula	65.3 ±2.1	47.6 ±1.8	66.2 ±2.2	0.99 ±0.06	86.5 ±0.7	42.2 ±2.2
Phi-3.5 zero-shot, direct	20.8 ±1.7	15.4 ±2.4	25.7 ±2.0	1.58 ±0.05	60.5 ±0.7	2.4 ±0.6
Phi-3.5 zero-shot, formula	43.5 ±2.3	26.8 ±1.7	25.7 ±2.0	2.08 ±0.08	60.5 ±0.7	2.4 ±0.6
Phi-3.5 + LoRA, formula	69.6 ±2.0	52.8 ±3.3	68.8 ±2.1	0.87 ±0.05	87.5 ±0.7	48.0 ±2.1

accuracy to 43.5%, still well below the fine-tuned model (69.6%) and TF-IDF (65.3%) on the same CVEs. Its score MAE gets worse, from 1.58 to 2.08, because its vectors are often wrong in ways that move the score a lot. For a model used zero-shot, formula scoring is first a consistency fix.

Table 5: **Fine-tuning makes outputs self-consistent.** Language-model outputs (%). *Score = formula*: the score the model wrote equals the CVSS v3.1 score of the vector it wrote. *Severity = tier of score*: the severity it wrote matches the tier of the score it wrote.

Model	Valid JSON	Score = formula	Severity = tier of score
Phi-3.5 zero-shot	99.7	3.4	37.0
Phi-3.5 + LoRA	100.0	99.8	100.0

5.5 Training and Inference Cost

Validation accuracy rose at every checkpoint, and the last one (step 5,000) was best (Figure 7). One more epoch might help further. Table 6 gives wall-clock times. Fine-tuning

Table 6: **Compute cost.** Wall-clock time on a single Apple M5 Pro (64 GB). Inference used small batches (at most 32 CVEs) and was not optimized for speed.

Step	Time
All TF-IDF baselines, tuning and test (CPU)	9 min
LoRA fine-tuning, 5,000 steps	2.5 h
LoRA inference, 36,325 test CVEs	4.9 h

took 2.5 hours and scoring the full test year 4.9 hours, against 9 minutes for every TF-IDF baseline together.

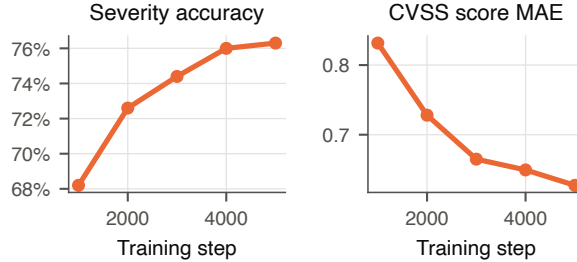


Figure 7: **Validation metrics still improve at the end of the first epoch.** Fine-tuned model with formula scoring, at each saved checkpoint, on 1,000 validation CVEs.

6 Limitations

Label noise. We treat NVD’s vector as ground truth, but analysts disagree, NVD revises records, and the organizations that report CVEs sometimes score them differently. Some of our “errors” are disagreements a human reviewer would accept. We pin the data snapshot so that our numbers stay fixed even as the labels change.

Single split and seed. We test on a single year and train each model once. The bootstrap intervals cover sampling of test CVEs, not variation from training. We fine-tune one 3.8B model for one epoch, and validation accuracy was still rising at the end (Figure 7). Larger models or longer training may do better. We did not run large hosted models, and we did not rebalance the training data for the fine-tuned model.

CVSS version. New CVEs increasingly carry CVSS v4.0 vectors. The formula approach carries over, since v4.0 also computes the score from the vector, but we did not test it.

Zero-shot sample. We evaluate the zero-shot model on 2,000 random test CVEs rather than all of them. The confidence intervals in Table 4 reflect this.

Input length. We use English descriptions and cut descriptions at 2,000 characters, which affects 1.4% of test CVEs.

7 Conclusion

A 3.8B-parameter model fine-tuned for 2.5 hours predicts every NVD enrichment field better than strong TF-IDF baselines on a full year of later CVEs. The margins are modest, and TF-IDF remains a fast, strong baseline that any new method should beat. Computing the score from the predicted vector keeps outputs consistent and helps a linear model, but it shifts errors toward over-calling CRITICAL, a trade-off that users should choose deliberately. The rarest severity tier, LOW, is still poorly predicted by every system. The next steps are rebalancing the training data, a second training epoch, larger open models, and CVSS v4.0.

References

- Marah Abdin et al. Phi-3 technical report: A highly capable language model locally on your phone, 2024.
- Masaki Aota, Hideaki Kanehara, Masaki Kubo, Noboru Murata, Bo Sun, and Takeshi Takahashi. Automation of vulnerability classification from its description using machine

- learning. In *IEEE Symposium on Computers and Communications (ISCC)*, 2020.
- Cédric Bonhomme and Alexandre Dulaunoy. VLAi: A RoBERTa-based model for automated vulnerability severity classification, 2025.
- Databricks. VulnWatch: AI-enhanced prioritization of vulnerabilities. <https://www.databricks.com/blog/vulnwatch-ai-enhanced-prioritization-vulnerabilities>, 2025.
- Clément Elbaz, Louis Rilling, and Christine Morin. Fighting n-day vulnerabilities with automated CVSS vector prediction at disclosure. In *Proceedings of the 15th International Conference on Availability, Reliability and Security (ARES)*, 2020. doi: 10.1145/3407023.3407038.
- FIRST. Common vulnerability scoring system v3.1: Specification document. <https://www.first.org/cvss/v3.1/specification-document>, 2019.
- Fraunhofer FKIE. NVD JSON data feeds, release v2026.10.05-000016. <https://github.com/fkie-cad/nvd-json-data-feeds>, 2026.
- Rikhiya Ghosh, Hans-Martin von Stockhausen, Martin Schmitt, George Marica Vasile, Sanjeev Kumar Karn, and Oladimeji Farri. CVE-LLM: Ontology-assisted automatic vulnerability evaluation using large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, 2025. doi: 10.1609/aaai.v39i28.35139.
- Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on apple silicon. <https://github.com/ml-explore/mlx>, 2023.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- Francesco Marchiori, Denis Donadel, and Mauro Conti. Can LLMs classify CVEs? investigating LLMs capabilities in computing CVSS vectors. In *TrustAI/CyberSec Workshop*, 2025.
- Nikita Mosievskiy. Fine-tuning RoBERTa for CVE-to-CWE classification: A 125M parameter model competitive with LLMs, 2026.
- National Institute of Standards and Technology. NIST updates NVD operations to address record CVE growth. <https://www.nist.gov/news-events/news/2026/04/nist-updates-nvd-operations-address-record-cve-growth>, April 2026. Accessed 2026-10-05.
- Fabian Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- Michael Roytman. Benchmarking LLMs on the vulnerability prioritization task. Empirical Security. <https://research.empiricalsecurity.com/research/benchmarking-llms>, 2025.
- Mustafizur R. Shahid and Hervé Debar. CVSS-BERT: Explainable natural language processing to determine the severity of a computer security vulnerability from its description. In *20th IEEE International Conference on Machine Learning and Applications (ICMLA)*, 2021.
- Rustam Talibzade, Idilio Drago, and Francesco Bergadano. On using LLMs for vulnerability classification. In *Proceedings of the 2025 Workshop on Large AI Systems and Models with Privacy and Security Analysis (LAMPS)*, 2025. doi: 10.1145/3733800.3763267.
- Mohammad Jalili Torkamani, Joey NG, Nikita Mehrotra, Mahinthan Chandramohan, Padmanabhan Krishnan, and Rahul Purandare. Streamlining security vulnerability triage with large language models, 2025.

A Prompts

Both prompts are sent as a single user turn through the Phi-3.5 chat template. Descriptions longer than 2,000 characters (1.4% of test CVEs) are cut at 2,000 characters.

Fine-tuned model (user turn).

You are a cybersecurity expert. Given the CVE description below, predict the NVD enrichment fields.
Return ONLY a JSON object with no extra text.

CVE Description:
<CVE description>

Zero-shot model (user turn).

You are a cybersecurity expert specializing in vulnerability analysis. Given a CVE description, predict the NVD CVSS v3.1 enrichment fields.

CVE Description:
<CVE description>

Respond ONLY with one JSON object with exactly these keys and allowed values:
{
 "attack_vector": "NETWORK"|"ADJACENT_NETWORK"|"LOCAL"|"PHYSICAL",
 "attack_complexity": "LOW"|"HIGH",
 "privileges_required": "NONE"|"LOW"|"HIGH",
 "user_interaction": "NONE"|"REQUIRED",
 "scope": "UNCHANGED"|"CHANGED",
 "confidentiality": "NONE"|"LOW"|"HIGH",
 "integrity": "NONE"|"LOW"|"HIGH",
 "availability": "NONE"|"LOW"|"HIGH",
 "cwe": "CWE-<number>",
 "cvss_score": <number 0.0-10.0>,
 "severity": "CRITICAL"|"HIGH"|"MEDIUM"|"LOW"}
}

B Reproducibility

All code, configuration files, and per-CVE predictions are in the project repository. The NVD snapshot is pinned to FKIE release v2026.10.05-000016. Each step below is one command and regenerates its part of the paper:

- data: download_data.py, prepare_splits.py;
- TF-IDF baselines: baselines_classical.py;
- LoRA training: mlx_lm.lora with scripts/lora_phi35.yaml;
- checkpoint selection: select_checkpoint.py;
- evaluation: evaluate_lora.py;
- tables, figures, and every number in the text: make_tables.py, plot_results.py.